Problem Statement : Design and implement Deep Convolutional GAN to generate images of faces/digits from a set of given images.

```
# Import necessary libraries
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np
import time

print(f"TensorFlow Version: {tf.__version__}")
```

```
TensorFlow Version: 2.19.0
```

```
# ==========================================
# Step 2: Prepare the dataset
# ==========================================
print("Loading and preprocessing the dataset...")
# Load the MNIST digits dataset
(train_images, train_labels), (_, _) =
tf.keras.datasets.mnist.load_data()

# Preprocess the images
# Reshape to (28, 28, 1) and normalize to the range [-1, 1]
# (tanh activation in the generator requires [-1, 1])
train_images = train_images.reshape(train_images.shape[0], 28, 28,
1).astype('float32')
train_images = (train_images - 127.5) / 127.5

# Batch and shuffle the data
BUFFER_SIZE = 60000
BATCH_SIZE = 256

train_dataset =
tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).
batch(BATCH_SIZE)
```

```
Loading and preprocessing the dataset...
```

```
# ==========================================
# Step 1: Initialize Generator and Discriminator
# ==========================================

def make_generator_model():
    model = tf.keras.Sequential()
    # Foundation for 7x7 image
    model.add(layers.Dense(7*7*256, use_bias=False,
input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
```

```
    model.add(layers.Reshape((7, 7, 256)))
    assert model.output_shape == (None, 7, 7, 256)

    # Upsample to 14x14
    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1),
padding='same', use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    # Upsample to 14x14
    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2),
padding='same', use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    # Upsample to 28x28 (Final Image Size)
    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2),
padding='same', use_bias=False, activation='tanh'))
    assert model.output_shape == (None, 28, 28, 1)

    return model

def make_discriminator_model():
    model = tf.keras.Sequential()
    # First Conv Layer
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2),
padding='same', input_shape=[28, 28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    # Second Conv Layer
    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2),
padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    # Flatten and Classify (Real or Fake)
    model.add(layers.Flatten())
    model.add(layers.Dense(1))

    return model

# Instantiate the models
generator = make_generator_model()
discriminator = make_discriminator_model()

# ==========================================
# Loss Functions and Optimizers
# ==========================================
```

```python
# Cross entropy loss
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    # Generator wants the discriminator to think the fake images are
real (1)
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)
```

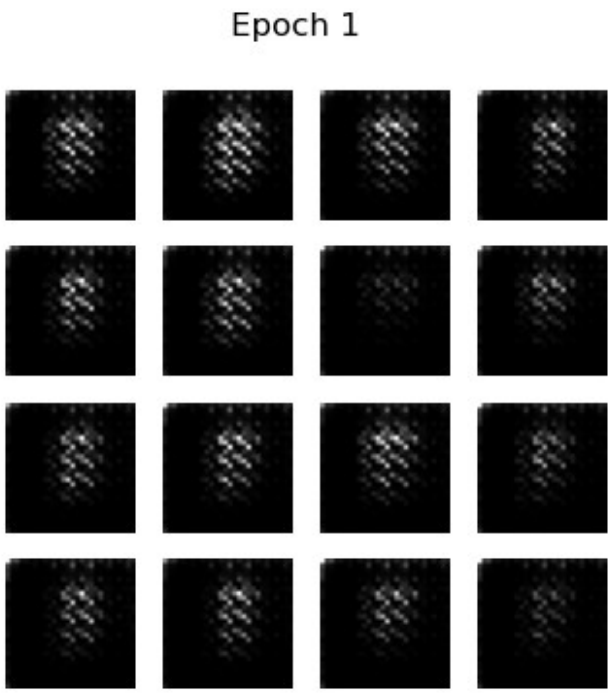
```
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\keras\
src\layers\core\dense.py:87: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
C:\Users\Admin\AppData\Roaming\Python\Python312\site-packages\keras\
src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```python
# ==========================================
# Steps 3, 4 & 5: Training Loop Details
# ==========================================
EPOCHS = 50
noise_dim = 100
num_examples_to_generate = 16

# Seed to keep the same generated images across epochs for
visualization
seed = tf.random.normal([num_examples_to_generate, noise_dim])

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as
disc_tape:
```

```
        # Generate images
        generated_images = generator(noise, training=True)

        # Evaluate real and fake images
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        # Step 3 & 4: Calculate losses
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    # Calculate gradients
    gradients_of_generator = gen_tape.gradient(gen_loss,
generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

    # Apply gradients (Update parameters)
    generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))

discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator
, discriminator.trainable_variables))

def generate_and_save_images(model, epoch, test_input):
    # Notice `training` is set to False.
    # This is so all layers run in inference mode (batchnorm).
    predictions = model(test_input, training=False)

    fig = plt.figure(figsize=(4, 4))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i+1)
        # Rescale from [-1, 1] to [0, 255] for plotting
        plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5,
cmap='gray')
        plt.axis('off')

    plt.suptitle(f"Epoch {epoch}")
    plt.show()

# Step 5: Iterate between Discriminator and Generator training
def train(dataset, epochs):
    print("Starting Training Loop...")
    for epoch in range(epochs):
        start = time.time()

        for image_batch in dataset:
            train_step(image_batch)
```

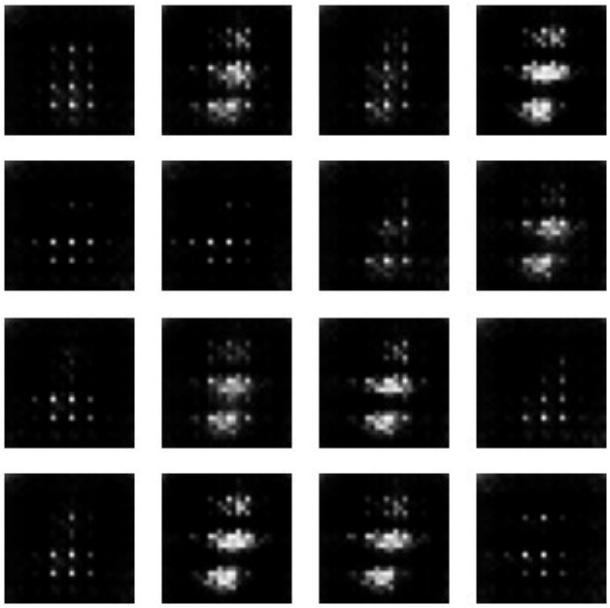
```
        # Produce images for the GIF/visualization as we go
        generate_and_save_images(generator, epoch + 1, seed)

        print(f'Time for epoch {epoch + 1} is {time.time()-start:.2f}
sec')

    # Generate final images
    generate_and_save_images(generator, epochs, seed)

# Run the training
train(train_dataset, EPOCHS)
```

```
Starting Training Loop...
```



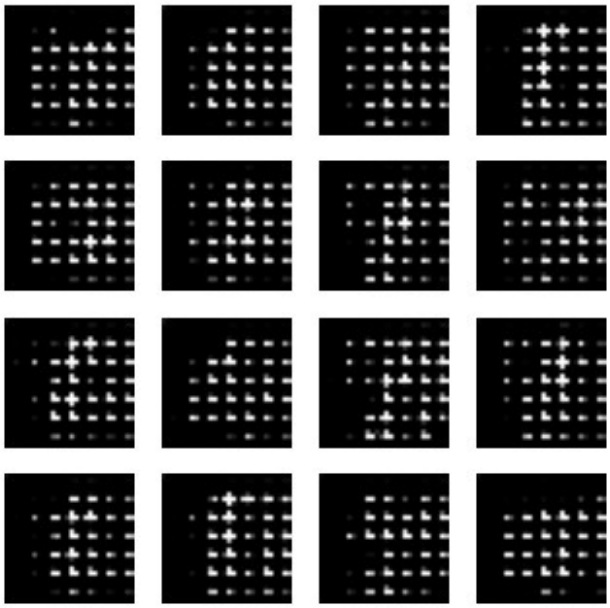

```
Time for epoch 1 is 150.86 sec
```

Epoch 2



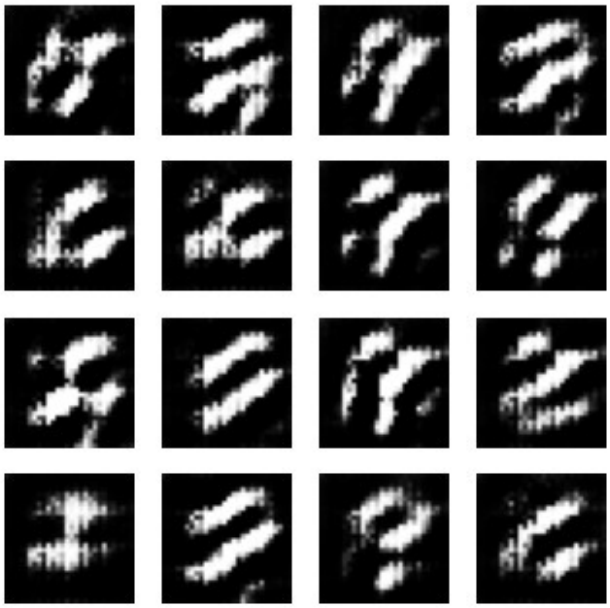
```
Time for epoch 2 is 163.39 sec
```

Epoch 3



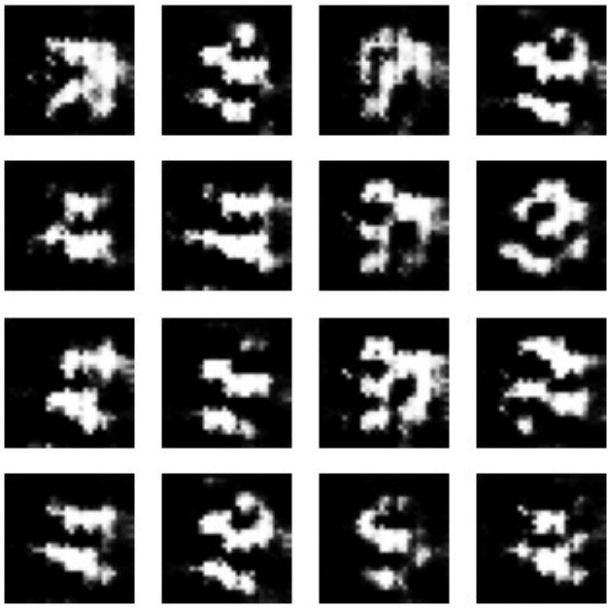
```
Time for epoch 3 is 169.66 sec
```

Epoch 4



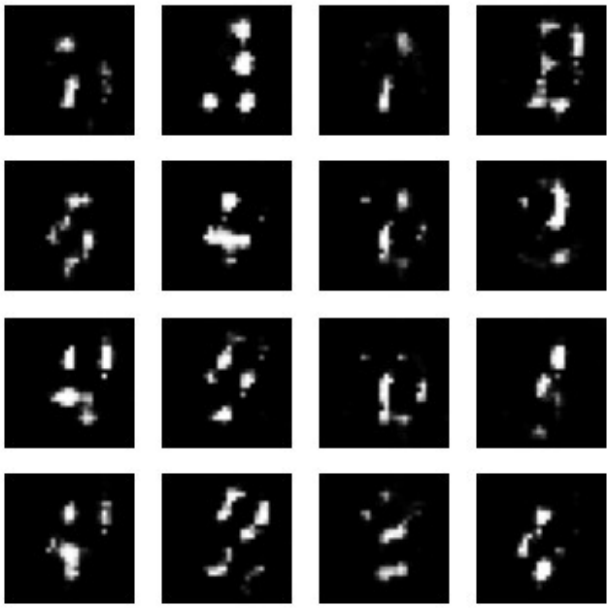
```
Time for epoch 4 is 172.97 sec
```

Epoch 5



```
Time for epoch 5 is 164.33 sec
```

Epoch 6

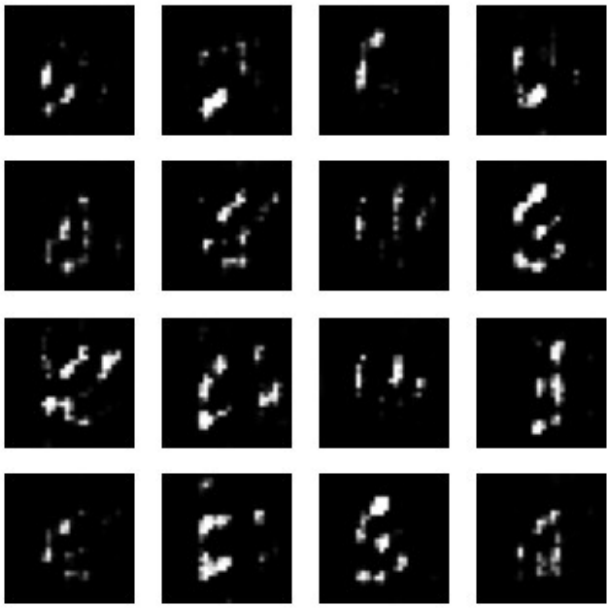


```
Time for epoch 6 is 166.59 sec
```

Epoch 7

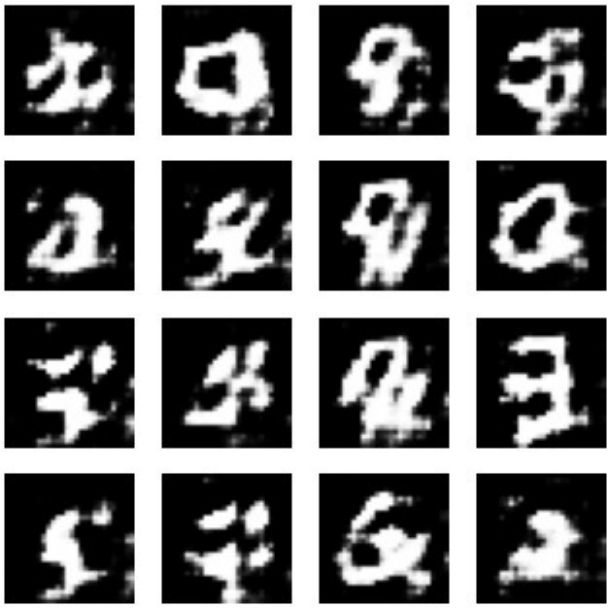
```
Time for epoch 7 is 165.75 sec
```

Epoch 8



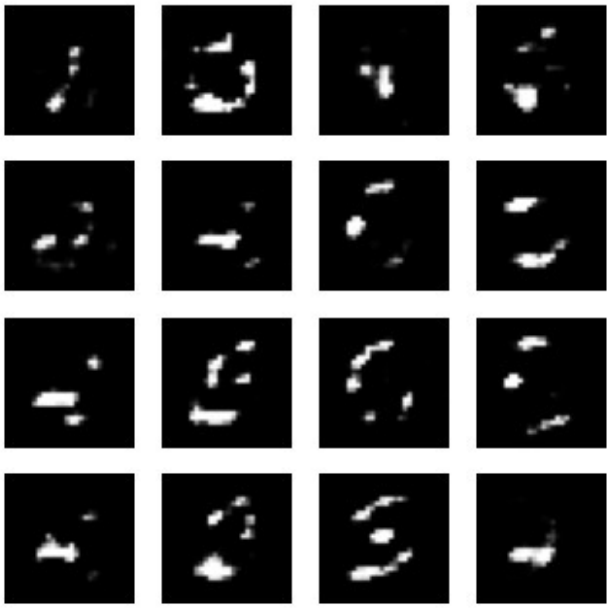
```
Time for epoch 8 is 169.71 sec
```

Epoch 9



```
Time for epoch 9 is 169.28 sec
```

Epoch 10

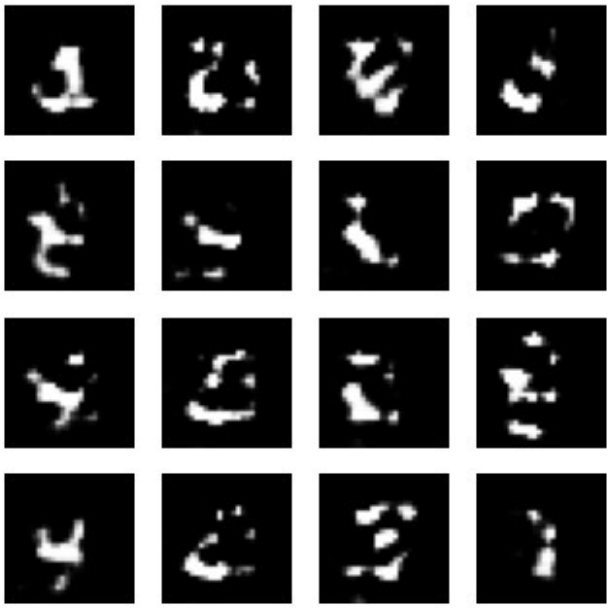


```
Time for epoch 10 is 165.97 sec
```

Epoch 11

```
Time for epoch 11 is 169.25 sec
```

Epoch 12

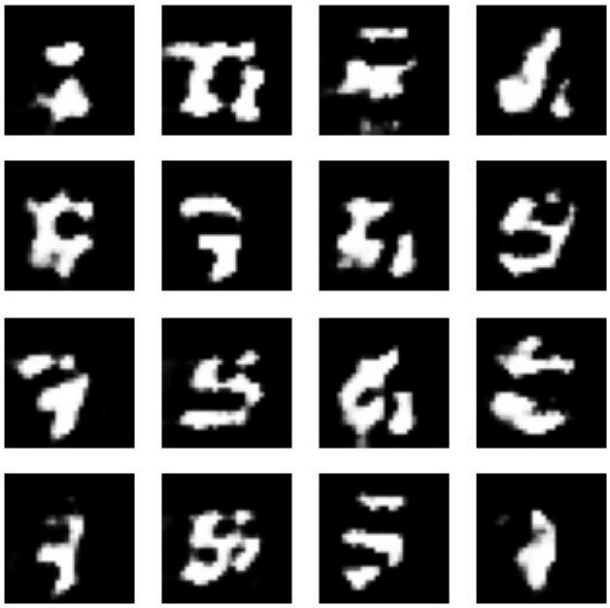


```
Time for epoch 12 is 169.03 sec
```

Epoch 13

```
Time for epoch 13 is 167.18 sec
```

Epoch 14



```
Time for epoch 14 is 166.93 sec
```

Epoch 15

```
Time for epoch 15 is 168.14 sec
```

Epoch 16

```
Time for epoch 16 is 166.19 sec
```

Epoch 17

```
Time for epoch 17 is 166.43 sec
```

Epoch 18

```
Time for epoch 18 is 166.61 sec

---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[4], line 69
     66     generate_and_save_images(generator, epochs, seed)
     68 # Run the training
---> 69 train(train_dataset, EPOCHS)

Cell In[4], line 58, in train(dataset, epochs)
     55 start = time.time()
     57 for image_batch in dataset:
---> 58     train_step(image_batch)
     60 # Produce images for the GIF/visualization as we go
     61 generate_and_save_images(generator, epoch + 1, seed)

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\python\util\traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    148 filtered_tb = None
    149 try:
--> 150   return fn(*args, **kwargs)
    151 except Exception as e:
    152   filtered_tb = _process_traceback_frames(e.__traceback__)

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
```

```
python\eager\polymorphic_function\polymorphic_function.py:833, in
Function.__call__(self, *args, **kwds)
    830 compiler = "xla" if self._jit_compile else "nonXla"
    832 with OptionalXlaContext(self._jit_compile):
--> 833   result = self._call(*args, **kwds)
    835 new_tracing_count = self.experimental_get_tracing_count()
    836 without_tracing = (tracing_count == new_tracing_count)

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\polymorphic_function\polymorphic_function.py:869, in
Function._call(self, *args, **kwds)
    866   self._lock.release()
    867   # In this case we have created variables on the first call,
so we run the
    868   # defunned version which is guaranteed to never create
variables.
--> 869   return tracing_compilation.call_function(
    870       args, kwds, self._no_variable_creation_config
    871   )
    872 elif self._variable_creation_config is not None:
    873   # Release the lock early so that multiple threads can
perform the call
    874   # in parallel.
    875   self._lock.release()

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\polymorphic_function\tracing_compilation.py:139, in
call_function(args, kwargs, tracing_options)
    137 bound_args = function.function_type.bind(*args, **kwargs)
    138 flat_inputs = function.function_type.unpack_inputs(bound_args)
--> 139 return function._call_flat(  # pylint: disable=protected-
access
    140     flat_inputs, captured_inputs=function.captured_inputs
    141 )

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\polymorphic_function\concrete_function.py:1322, in
ConcreteFunction._call_flat(self, tensor_inputs, captured_inputs)
   1318 possible_gradient_type =
gradients_util.PossibleTapeGradientTypes(args)
   1319 if (possible_gradient_type ==
gradients_util.POSSIBLE_GRADIENT_TYPES_NONE
   1320     and executing_eagerly):
   1321   # No tape is watching; skip to running the function.
-> 1322   return self._inference_function.call_preflattened(args)
   1323 forward_backward =
self._select_forward_and_backward_functions(
   1324     args,
   1325     possible_gradient_type,
   1326     executing_eagerly)
```

```
   1327 forward_function, args_with_tangents =
forward_backward.forward()

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\polymorphic_function\atomic_function.py:216, in
AtomicFunction.call_preflattened(self, args)
    214 def call_preflattened(self, args: Sequence[core.Tensor]) ->
Any:
    215   """Calls with flattened tensor inputs and returns the
structured output."""
--> 216   flat_outputs = self.call_flat(*args)
    217   return self.function_type.pack_output(flat_outputs)

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\polymorphic_function\atomic_function.py:251, in
AtomicFunction.call_flat(self, *args)
    249 with record.stop_recording():
    250   if self._bound_context.executing_eagerly():
--> 251     outputs = self._bound_context.call_function(
    252         self.name,
    253         list(args),
    254         len(self.function_type.flat_outputs),
    255     )
    256   else:
    257     outputs = make_call_op_in_graph(
    258         self,
    259         list(args),
    260         self._bound_context.function_call_options.as_attrs(),
    261     )

File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\context.py:1688, in Context.call_function(self, name,
tensor_inputs, num_outputs)
   1686 cancellation_context = cancellation.context()
   1687 if cancellation_context is None:
-> 1688   outputs = execute.execute(
   1689       name.decode("utf-8"),
   1690       num_outputs=num_outputs,
   1691       inputs=tensor_inputs,
   1692       attrs=attrs,
   1693       ctx=self,
   1694   )
   1695 else:
   1696   outputs = execute.execute_with_cancellation(
   1697       name.decode("utf-8"),
   1698       num_outputs=num_outputs,
   (...)
   1702       cancellation_manager=cancellation_context,
   1703   )
```

```
File ~\AppData\Roaming\Python\Python312\site-packages\tensorflow\
python\eager\execute.py:53, in quick_execute(op_name, num_outputs,
inputs, attrs, ctx, name)
     51 try:
     52   ctx.ensure_initialized()
---> 53   tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle,
device_name, op_name,
     54                                        inputs, attrs,
num_outputs)
     55 except core._NotOkStatusException as e:
     56   if name is not None:

KeyboardInterrupt:
```